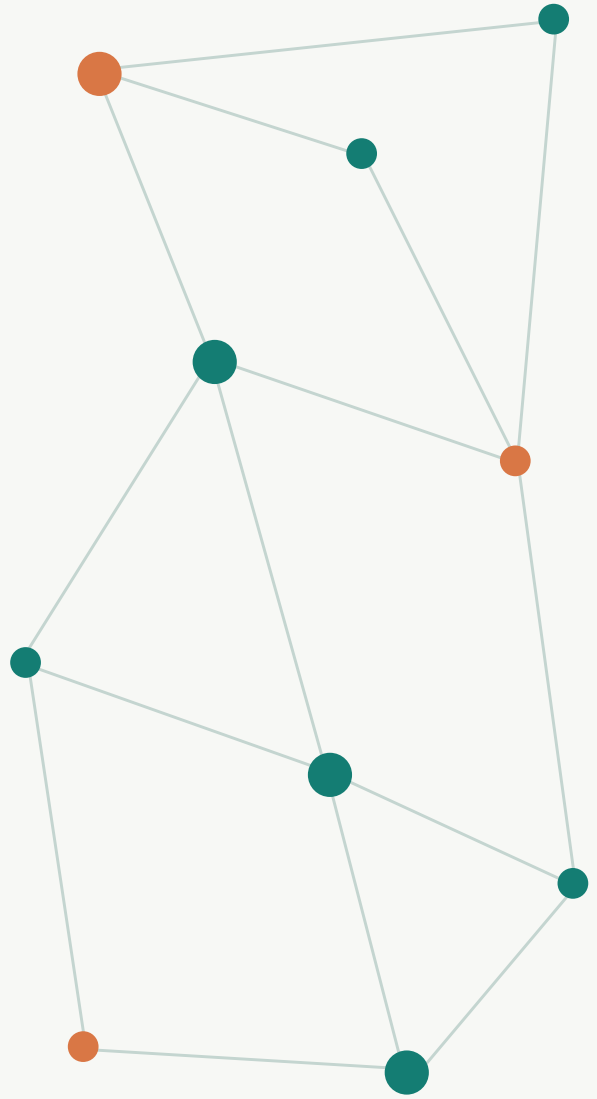
NEWTON SCHOOL OF TECHNOLOGY TALK

OPEN SOURCE

IN THE AGE OF AI

If AI can write the code, what makes open-source valuable?

A maintainer's perspective on contribution, trust, and what comes next.



CONTRIBUTION · COMMUNITY · CODE QUALITY · FUTURE

Vinit Shahdeo | Technical Lead & 3x GitHub Star

If AI Can Write Code, Is Open Source Dead?

Start with the question students may already be asking.

AI can generate

CODE

AI can generate

TESTS

AI can assist

REVIEWS

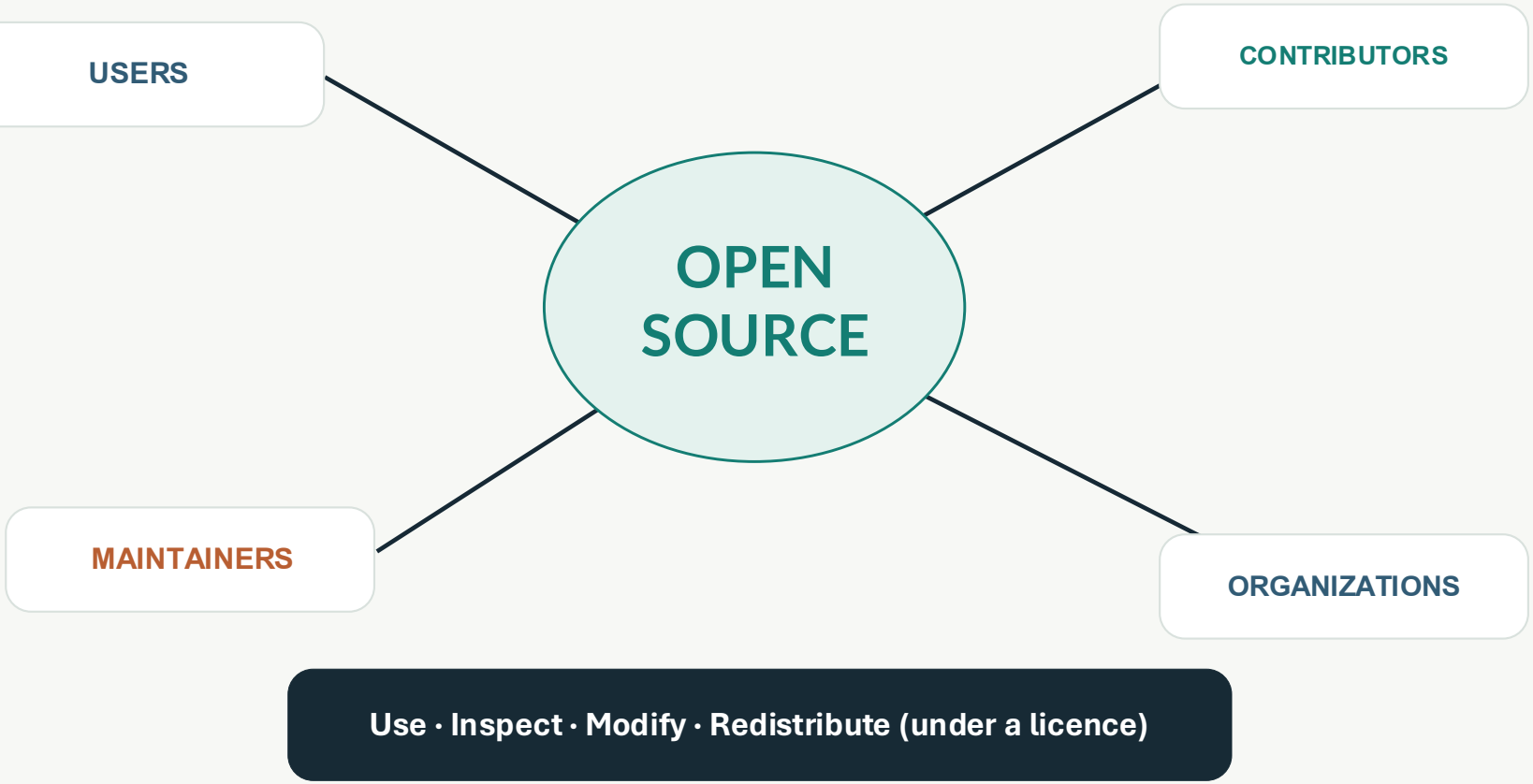
So, what still requires people?

- ❑ Deciding what should be built
- ❑ Verifying what is correct
- ❑ Maintaining trust over time

Vote: YES / NO / NOT SURE

Open Source Is More Than Code

An ecosystem of rights, collaboration, and long-term stewardship.



Why Should Students Contribute?

Real-world practice that goes beyond classroom projects.

01

REAL CODEBASES

Tests, bugs, APIs, compatibility

02

COLLABORATION

Feedback, async work, shared decisions

03

PUBLIC EVIDENCE

Concrete work to discuss in interviews

04

REAL IMPACT

Changes used beyond the classroom

05

ENGINEERING JUDGEMENT

Learn trade-offs, not just syntax

THE POINT

Show how you work when you don’t own the codebase.

Your Digital Footprint Is Your New Resume

Your work online tells a story before your CV does.

01

BUILD IN PUBLIC

Projects, pull requests, docs, and write-ups.

02

SHOW CONSISTENCY

Small, meaningful contributions over time.

03

MAKE IT FINDABLE

Pin strong repos. Explain your role and impact.

Maintain a GitHub streak like you maintain a Snapchat streak.

Consistency over vanity commits. Quality still wins.

A public portfolio helps people see how you think—not just what you claim.



Learn more: digitalfootprintbook.com

Four Myths That Hold Beginners Back

A small, useful contribution beats a perfect plan to start later.

“I must be an expert.”	Start with a scoped problem.
“Only code counts.”	Docs, tests, triage, and support matter.
“Rejection means failure.”	Review is part of learning.
“I need GSoC first.”	Start contributing before applying.

A Practical Roadmap to Your First PR

Start with understanding. Build confidence through small iterations.

01
CHOOSE
A project you already use

02
UNDERSTAND
Guide, code, tests, issue

03
SCOPE
One real, small problem

04
SUBMIT
Focused PR + evidence

05
ITERATE
Learn from review

Goal: understand one real problem—not simply get merged fast.

From Contributor to GSoC or LFX Mentee

Programmes can deepen an existing journey—not replace it.

PROGRAMME 01

Google Summer of Code

Mentored open-source projects with a defined scope, proposal, timeline, and timeline, and evaluation.

PROGRAMME 02

LFX Mentorship

Term-based mentorship opportunities across participating Linux Foundation communities.

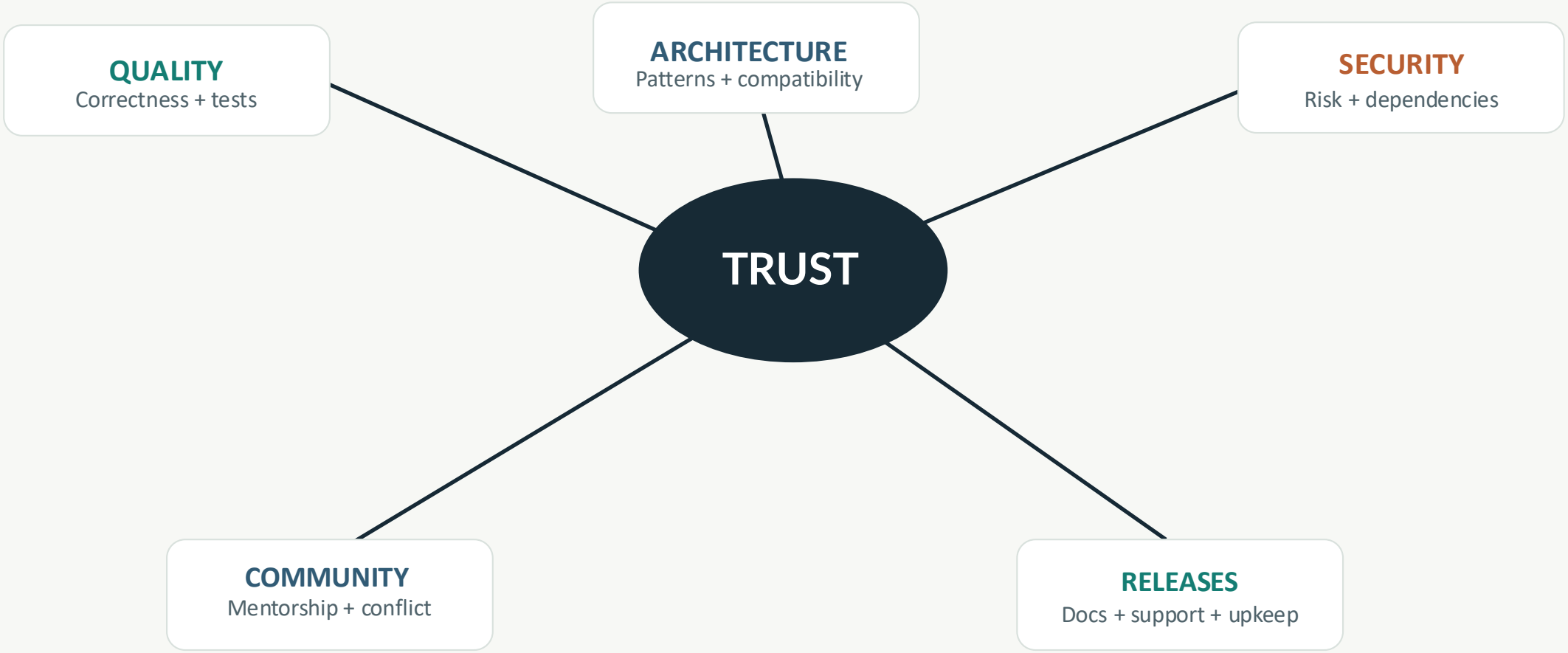
THE PATH

Explore → Join → Contribute → Propose → Apply

Verify eligibility, current openings, and deadlines on official sites.

A Maintainer’s Job Is Bigger Than Reviewing Code

Every accepted change creates a future responsibility.



What Makes a Pull Request Good?

The aim is not just code that runs—it is a change the project can trust.

01	REAL PROBLEM	Tied to an issue or agreed need
02	FOCUSED SCOPE	No unrelated refactors
03	MEANINGFUL TESTS	Expected behaviour + edge cases
04	PROJECT FIT	Design and conventions respected
05	CLEAR EXPLANATION	What changed, why, how verified

Passing tests help—but do not guarantee good design or project fit.

What Keeps Contributors Coming Back?

A first merged PR is a moment. A healthy community is a reason to stay.

PURPOSE

Work that matters to real users

MASTERY

Learn systems and engineering judgement

BELONGING

Respectful relationships and feedback

GROWTH

Path from contributor to reviewer

Burnout risks

- Unrealistic expectations
- Hostile reviews
- Hidden maintainer workload

Mentorship should build independence—not dependence.

AI Changes the Economics of Contribution

The cost of creating a patch is falling faster than the cost of reviewing one.

PRODUCTION

Lower friction

AI can explain code, draft fixes, generate tests, and and propose changes.

VERIFICATION

Still needs context

Someone must check correctness, security, scope, and long-term impact.

More code ≠ more useful contributions

AI in Open Source: The Latest Numbers

Linux Foundation Research · published 20 October 2026

52%

Less time spent authoring code

Estimated average reduction

25%

Incoming PRs considered low-quality

Described as “AI slop”

59%

Maintainers report rising contributions

Since GenAI became widespread

35%

Projects permit GenAI formally

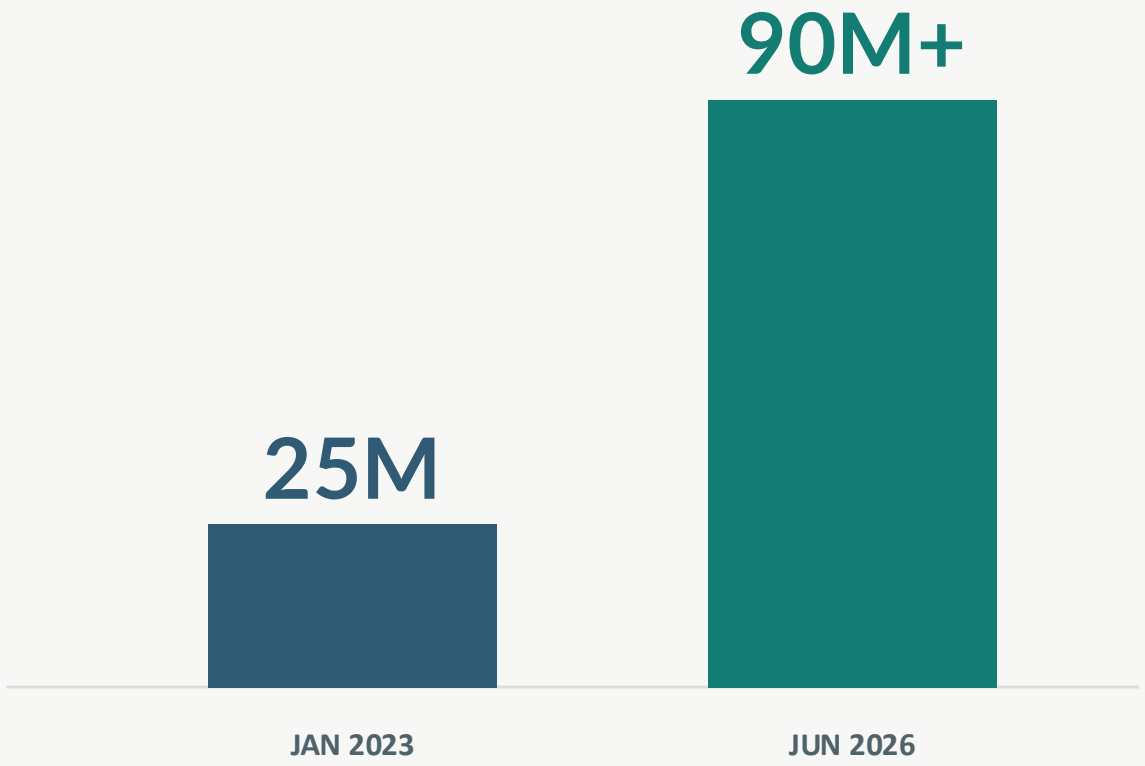
Formal policy reported by projects

Survey estimates and reported perceptions—not universal rates across all projects.

The Review Queue Is Growing

GitHub reports a roughly 3.6× rise in monthly merged pull requests since January 2023.

Merged pull requests per month across GitHub



≈ 3.6×

monthly merged PR volume

Growth reflects the wider ecosystem—not AI alone.

Implication: the cost to create can outrun the capacity to review.

Can AI Help Review the Code It Helps Produce?

GitHub Copilot code review: product-reported figures, March 2026.

60M+

code reviews and counting

More than 1 in 5 code reviews on GitHub involved involved Copilot code review.

Actionable feedback per review



29% no comments

DESIGN PRINCIPLE

Signal over volume

Useful findings matter more than more comments.

GitHub-reported product data—not an independent quality benchmark.

Code That Works Is Not Always Good Code

Compilation is one gate—not the finish line.

GENERATES	Syntax and plausible implementation
COMPILES	Basic correctness
PASSES TESTS	Expected cases and regressions
PRODUCTION READY	Security · reliability · maintainability

Potential gaps: edge cases · security · API contracts · unnecessary complexity

Will Specs Become More Important Than Code?

A plausible shift: define intent clearly, then generate and verify the implementation.

01
SPECIFY
What and why

→

02
CLARIFY
Resolve ambiguity

→

03
PLAN
Architecture + constraints

→

04
IMPLEMENT
AI-assisted code

→

05
VERIFY
Tests + convergence

SPEC = SOURCE OF INTENT

Requirements · constraints · acceptance criteria

CODE = EXECUTABLE ARTIFACT

Still needs tests, review, and maintenance

Prediction, not replacement: specs may guide more work, but code remains essential.

Why Are Projects Becoming Stricter About AI?

The concern is not AI itself. It is quality, capacity, and accountability.

35%

Formal policy permitting GenAI

32%

No policy, with no plans for one

53%

Disagree AI improved community trust

CAPACITY

Review queues grow

SAFETY

Changes need verification

GOVERNANCE

Policies set expectations

Stricter policy can mean disclosure, testing, data safeguards—or restrictions.

Use AI Responsibly: The Contributor's Checklist

AI can assist the work. The contributor remains accountable.

01	UNDERSTAND FIRST	Read the issue and investigate the code.
02	FOLLOW POLICY	Check project rules and protect sensitive data.
03	VERIFY OUTPUT	Review the diff, edge cases, and tests.
04	BE TRANSPARENT	Disclose AI assistance when required.
05	TAKE OWNERSHIP	Explain and stand behind every change.

Use AI to learn—not to bypass understanding.

Use AI as a Pair Programmer, Not an Autopilot

Better context + smaller tasks + verification = more useful assistance.

CONTEXT

Issue, relevant files, project conventions

TASK

One clear outcome; ask for a plan first

CONSTRAINTS

Scope, compatibility, security, style

VERIFICATION

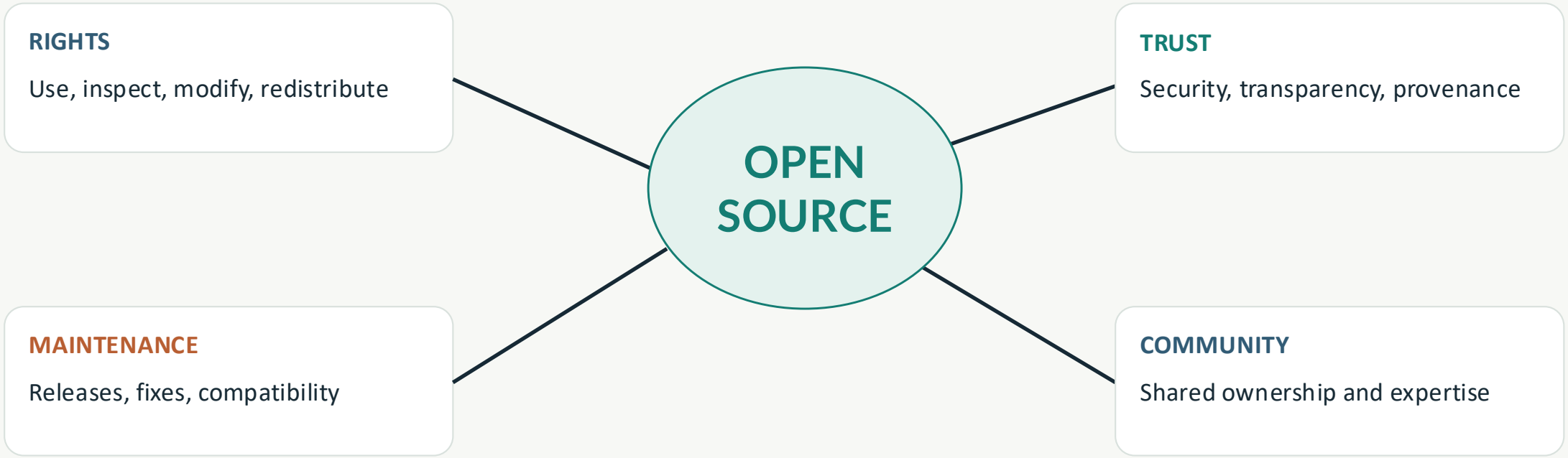
Tests, edge cases, explain the diff

“First propose a plan. Keep the change focused, add tests for edge cases, and explain the diff and test results.”

Prompt recipe: Context → Task → Constraints → Verification

So, Will Open-Source Die?

No—but the value may shift from producing code to sustaining trustworthy software.



Code generation ≠ sustainable software

AI Is Creating New Open-Source Ecosystems

Not just code written by AI—new software ecosystems people can build together.

01 MODELS & DATASETS

Open models, model tooling, datasets, and documentation.

02 SKILLS & AI AGENTS

Reusable skills, agent workflows, tools, and integrations.

03 LIBRARIES & DEV TOOLS

SDKs, frameworks, CLIs, plugins, and developer experience.

04 INFERENCE & SERVING

Inference engines, runtimes, deployment, and model serving.

05 DATA & EVALUATION

Data pipelines, benchmarks, evaluations, and observability.

06 SAFETY & SECURITY

Red-teaming, guardrails, privacy, and vulnerability research.

AI doesn't shrink the open-source map. It creates new territory.

Build it. Test it. Document it. Secure it. Maintain it—together.

More Automation. More Need for Judgement.

A forecast—not a certainty. Communities will shape which future arrives.

AUTOMATION

- Routine fixes and triage
- Draft tests and docs
- Dependency updates

HUMAN JUDGEMENT

- Security and risk decisions
- Architecture and trade-offs
- Trust, mentorship, funding

Future advantage = productive tools + healthy communities.

What Should Future Contributors Learn?

Shift from producing lines of code to delivering reliable software.

AI CAN HELP WITH

- Draft routine functions
- Explain unfamiliar code
- Suggest fixes and tests
- Generate documentation

HUMANS MUST DEVELOP

- Understand systems and requirements
- Debug and find root causes
- Design meaningful tests
- Review architecture and security
- Communicate and take ownership

Fundamentals matter more when you are responsible for verifying the output.

You Are the Maintainer: What Would You Do?

A realistic review dilemma—there is more to it than “AI or no AI”.

SCENARIO

An AI-assisted PR fixes a real issue.

500 lines · no tests · contributor cannot explain parts of the implementation

A	MERGE
B	REQUEST CHANGES
C	CLOSE

Show of hands → What evidence would change your decision?

Three Things to Remember

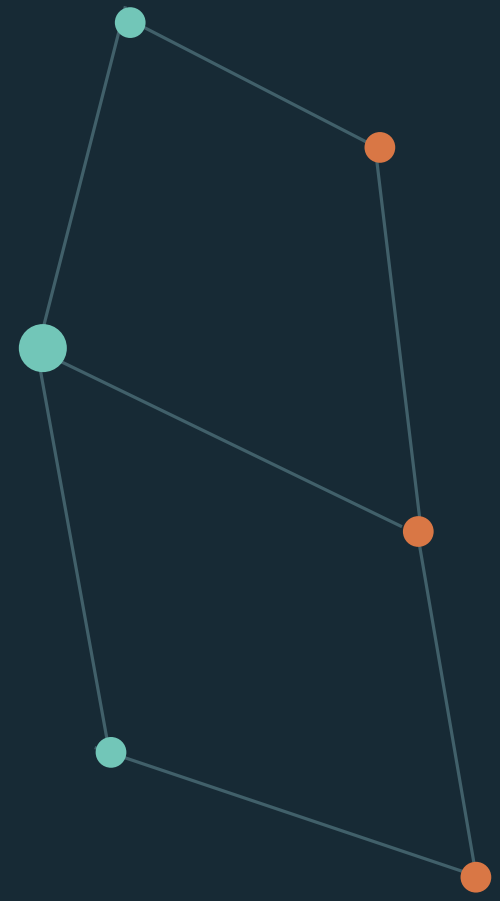
If students remember only three points, make them these.

01	START SMALL	Solve one real problem.
02	EARN TRUST	Be clear, test your work, learn from review.
03	USE AI CRITICALLY	Accelerate learning; never outsource accountability.

When code becomes abundant, judgement becomes valuable.

OPEN DISCUSSION

LET'S DISCUSS.



Questions for the room

- What stops you from contributing today?
- Should AI use always be disclosed?
- How should maintainers handle low-quality PRs?
- What makes you stay in a community?

THANK YOU

Let’s connect.

Keep the conversation going beyond this talk.

FIND ME ONLINE

GH	GITHUB	github.com/vinitshahdeo
in	LINKEDIN	linkedin.com/in/vinitshahdeo
X	TWITTER / X	@Vinit_Shahdeo

Vinit Shahdeo · Technical Lead · 3× GitHub Star



Scan and subscribe for more